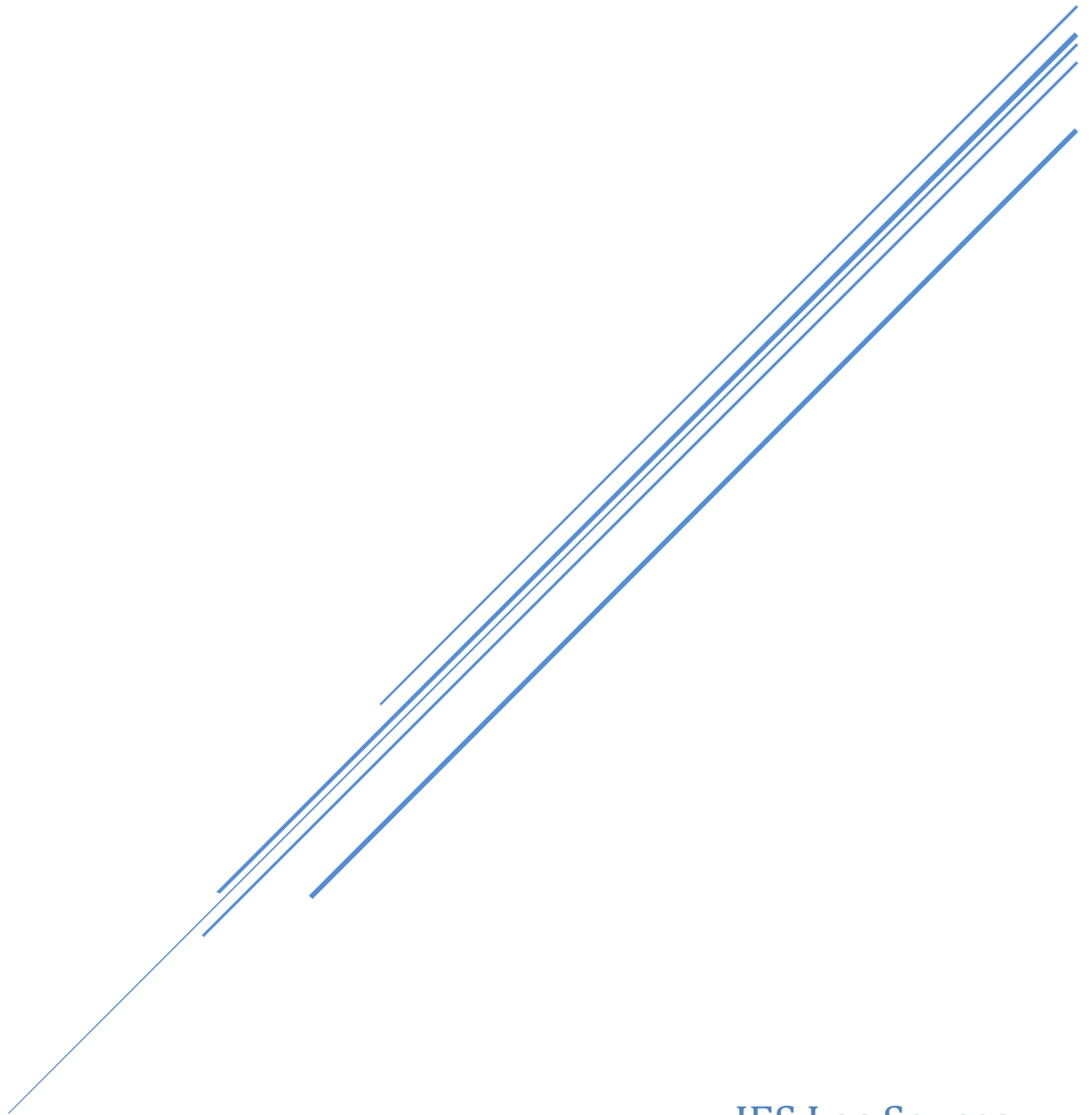


DESARROLLO WEB EN ENTORNO SERVIDOR

Ejercicios del tema 1



IES Los Sauces
DAW2

Contenido

1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS.....	2
2. Modelo de comunicaciones cliente – servidor y su relación con las aplicaciones web.	2
3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados.....	3
4. Estudio sobre el concepto de URI.....	3
5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa	4
6. Modelo de división funcional front-end / back-end para aplicaciones web	4
7. Página web estática – página web dinámica – aplicación web – mashup	5
8. Componentes de una aplicación web.	6
9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor.....	6
10. Lenguajes de programación utilizados en el lado servidor de una aplicación web	7
11. Características y posibilidades de desarrollo de una plataforma XAMPP	8
12. En qué casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entorno de explotación.....	8
13. IDE más utilizados (características y grado de implantación actual).	9
14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual).....	10
15. Apache HTTP vs Apache Tomcat	10
16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual).....	11

1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS.

IP: es el protocolo de Internet que dirige los paquetes de datos entre dispositivos en una red, identificando su origen y destino.

TCP: garantiza que los datos lleguen completos y en orden, estableciendo una conexión fiable.

HTTP: permite la comunicación entre navegadores y servidores para acceder a páginas web.

HTTPS: es la versión segura de HTTP, que cifra la comunicación mediante SSL/TLS para proteger la información.

El protocolo IP (*Internet Protocol*), perteneciente a la capa de red, se encarga de transmitir los datos enviándolos de enlace en enlace. Por su parte, el protocolo TCP (*Transmission Control Protocol*), situado en la capa de transporte, ofrece un servicio de comunicación fiable, bidireccional (dúplex) y orientado a la conexión. Finalmente, en la capa de aplicación del modelo TCP/IP encontramos el protocolo HTTP, utilizado para la comunicación entre cliente y servidor. Su versión segura, HTTPS, cifra esta comunicación para garantizar que la información sea completamente ilegible para terceros.

2. Modelo de comunicaciones cliente – servidor y su relación con las aplicaciones web.

El modelo cliente-servidor es la arquitectura base de las aplicaciones web. Funciona mediante la interacción entre el navegador del usuario (cliente) y el servidor, que procesa y devuelve la información solicitada.

Esta comunicación puede darse de las siguientes formas:

Comunicación síncrona: El cliente envía una petición mediante protocolos como HTTP o HTTPS y debe esperar la respuesta del servidor para poder continuar.

Comunicación asíncrona: El cliente envía peticiones en segundo plano y el servidor devuelve datos (normalmente en JSON o XML) sin que la página necesite recargarse por completo, aportando un gran dinamismo.

Comunicación en tiempo real: Cliente y servidor mantienen un canal abierto para enviar y recibir información de forma continua, sin necesidad de generar nuevas solicitudes.

3. Estudio sobre los métodos de petición HTTP /HTTPS más utilizados.

Estos son los principales métodos de petición HTTP y sus funciones (*he corregido un pequeño error tipográfico en "POST"*):

GET: Recupera información del servidor sin alterarla. Es el método estándar para consultar datos en páginas web o APIs.

POST: Envía datos al servidor (como formularios o archivos) para que sean procesados. Esta acción puede modificar el estado del servidor, creando nuevos registros o actualizando los actuales.

PUT: Actualiza un recurso específico o lo crea de cero en caso de que no exista. Es muy utilizado en aplicaciones que gestionan bases de datos.

DELETE: Elimina un recurso específico almacenado en el servidor.

HEAD: Funciona igual que GET, pero solo solicita los encabezados de la respuesta (no descarga el cuerpo del mensaje).

4. Estudio sobre el concepto de URI

Un Identificador de Recursos Uniforme (URI) permite identificar de manera unívoca recursos físicos o abstractos en una red. Todo URI puede estar compuesto por una URL, un URN, o ambos:

URL (Localizador): Secuencia de caracteres que permite ubicar y recuperar información en internet especificando su punto de acceso (frecuentemente vinculada a protocolos HTTP/HTTPS).

URN (Nombre): Identificador global y único que nombra al recurso independientemente de su localización (por ejemplo, el código ISBN de un libro).

Estructura de un URI:

Esquema: Define la especificación del identificador o el protocolo de acceso al recurso (ej. *https*).

Autoridad: Elemento jerárquico que señala al servidor o dominio responsable.

Ruta: Información jerárquica que traza el camino exacto hacia el recurso dentro del servidor.

Consulta: Parámetros o datos con estructura no jerárquica que ayudan a filtrar o especificar el recurso.

Fragmento: Indicador que dirige a una parte concreta o vista secundaria dentro del recurso principal.

5. Modelo de desarrollo de aplicaciones multicapa – comunicación entre capas – componentes – funcionalidad de cada capa

Una aplicación multicapa se basa en un diseño de software que divide el sistema en niveles independientes, asignando una responsabilidad única a cada uno. Este enfoque destaca por mejorar la modularidad, facilitar el mantenimiento y potenciar la escalabilidad.

El flujo de comunicación es jerárquico y unidireccional: cada capa interactúa exclusivamente con la inmediatamente adyacente (superior o inferior).

Sus capas principales son:

Capa de presentación: Gestiona la interfaz y se encarga de mostrar la información al usuario.

Capa de lógica de negocio: Define las reglas y ejecuta las operaciones que determinan el funcionamiento del sistema.

Capa de datos: Actúa como puente con la base de datos para ejecutar consultas, inserciones y actualizaciones.

Capa de persistencia: Garantiza el almacenamiento y la gestión estable de la información a largo plazo.

6. Modelo de división funcional front-end / back-end para aplicaciones web

El desarrollo de aplicaciones web se divide en dos entornos de trabajo fundamentales:

Front-end (Interfaz de usuario): Es la parte visible y accesible del sitio web con la que el usuario interactúa directamente.

Funciones: Define el diseño, la estructura visual y la funcionalidad de la interfaz.

Tecnologías: Utiliza lenguajes como HTML, CSS y JavaScript para la creación y el estilizado.

Objetivo: Optimizar la experiencia del usuario (UX) para que la navegación resulte intuitiva y atractiva.

Dinámica: Se encuentra en constante evolución para adaptarse a las nuevas tendencias tecnológicas y de diseño.

Back-end (Lógica del servidor): Es la parte invisible para el usuario final, pero resulta indispensable para el funcionamiento operativo de la plataforma.

Funciones: Procesa y almacena los datos, gestiona la lógica de negocio, administra la base de datos y se comunica con el front-end.

Tecnologías: Emplea lenguajes de programación como PHP, Python, Ruby o Java.

Responsabilidad: Es el encargado de garantizar la seguridad, el rendimiento (eficacia) y la escalabilidad de todo el sistema.

7. Página web estática – página web dinámica – aplicación web – mashup

Página web estática: Muestra un contenido fijo que es idéntico para todos los usuarios cada vez que acceden. Se desarrolla únicamente con tecnologías de cliente (HTML, CSS, JavaScript) y no necesita bases de datos ni procesamiento en el servidor.

Página web dinámica: Adapta y cambia su contenido en tiempo real según la interacción del usuario, su ubicación u otros factores. Se apoya en lenguajes como PHP o JavaScript y en bases de datos para generar información personalizada.

Aplicación web: Es un software complejo que se ejecuta directamente en el navegador, evitando que el usuario instale programas adicionales. Gracias a su enorme accesibilidad, las empresas las utilizan masivamente para ofrecer servicios, intercambiar información y comunicarse de manera segura con sus clientes.

Mashup: Aplicación o página web que combina datos y funcionalidades procedentes de múltiples fuentes externas (generalmente a través de APIs, como Google Maps integrado en una web inmobiliaria) para crear un servicio nuevo y unificado.

8. Componentes de una aplicación web.

1. El Frontend (Lado del cliente)

Es la parte visible e interactiva con la que el usuario interactúa directamente en su navegador web.

Interfaz de usuario (UI): Botones, formularios, menús, imágenes y textos.

Tecnologías principales: HTML para la estructura, CSS para el diseño y los estilos, y JavaScript para el comportamiento y la interactividad en tiempo real.

2. El Backend (Lado del servidor)

Es el "cerebro oculto" que procesa la lógica de la aplicación, gestiona las peticiones del usuario y coordina la comunicación entre el navegador y los datos.

Servidor web: Recibe y administra las peticiones HTTP (por ejemplo, Apache, Nginx).

Lógica de negocio: El código y los scripts que determinan cómo se procesan los datos, las reglas de la aplicación y la autenticación de usuarios (con lenguajes como Node.js, Python, PHP o Java).

3. La Base de datos

Es el almacén estructurado donde se guarda de forma persistente toda la información de la aplicación (como perfiles de usuario, catálogos de productos o registros).

Sistemas de gestión: Permiten consultar, modificar y organizar los datos (por ejemplo, MySQL, PostgreSQL, MongoDB).

9. Programas ejecutados en el lado del cliente y programas ejecutados en el lado del servidor

El funcionamiento de una aplicación web utiliza tecnologías diferentes dependiendo de dónde se ejecuten los procesos:

Ejecución en el lado del cliente (Frontend): El programa encargado de ejecutar las aplicaciones en este entorno es el **navegador web**. Sus tecnologías principales son:

HTML: Define la estructura y el contenido básico de la página.

CSS: Aplica los estilos y la apariencia visual.

JavaScript: Aporta la lógica y la interactividad.

Frameworks JS: Herramientas (como React o Angular) utilizadas para construir aplicaciones dinámicas y complejas.

WebAssembly: Tecnología que permite ejecutar código de otros lenguajes de programación directamente en el navegador.

Ejecución en el lado del servidor (Backend): En este entorno se ejecutan los servidores web, los intérpretes de código y los sistemas de bases de datos. Sus componentes principales son:

Servidores web: Programas que gestionan las peticiones. Destacan **Apache** (el más clásico, estable y configurable) y **Nginx** (muy eficiente e ideal para alto tráfico).

Lenguajes y Frameworks: Se utilizan lenguajes robustos para gestionar la lógica de negocio y las peticiones al servidor. Ejemplos destacados son **PHP** (con su framework *Laravel*), **Python** (con *Django*) o **Java** (con *Spring Boot*).

Bases de datos: Se utiliza el lenguaje **SQL** para realizar consultas y modificaciones, operando sobre sistemas de gestión de bases de datos (SGBD) como **MySQL** o **PostgreSQL**.

10. Lenguajes de programación utilizados en el lado servidor de una aplicación web

Estos son los principales lenguajes de programación utilizados en el lado del servidor, junto con sus características clave:

PHP:

Lenguaje de código abierto, interpretado, orientado a objetos y de tipado débil (no es necesario especificar el tipo al declarar variables). Permite una fácil integración de módulos externos. Es uno de los lenguajes más consolidados y utilizados en la web actual.

Ruby:

Lenguaje interpretado y orientado a objetos. Es muy dinámico y flexible (permite modificar su estructura durante la ejecución) y destaca por su amplio ecosistema de "gemas" (paquetes de código reutilizable).

Java:

Lenguaje robusto, orientado a objetos y fuertemente tipado. Es un lenguaje compilado que se ejecuta a través de la Máquina Virtual de Java (JVM). Muy extendido en aplicaciones empresariales y sistemas de gran escala.

JavaScript:

Lenguaje interpretado, de tipado débil y orientado a objetos (basado en prototipos). Funciona de manera asíncrona y basada en eventos. Extremadamente versátil y popular hoy en día, ya que permite programar tanto el lado del cliente como el del servidor (usando entornos como Node.js).

11. Características y posibilidades de desarrollo de una plataforma XAMPP

XAMPP es un paquete de software libre diseñado para instalar y configurar fácilmente un entorno de desarrollo web local. Su propósito es permitir a los desarrolladores crear, probar y administrar aplicaciones directamente en sus equipos antes de publicarlas.

Componentes que integra: El paquete incluye todo lo necesario para el backend: el servidor web **Apache**, el gestor de bases de datos **MySQL**, y los lenguajes de programación **PHP** y **Perl**.

Características principales:

- **Paquete integral:** Proporciona en una sola descarga las herramientas clave (servidor, base de datos y lenguajes) para desarrollar aplicaciones web.
- **Instalación simplificada:** Permite montar un entorno de trabajo completo y funcional en unos pocos pasos.
- **Funcionamiento offline:** No requiere conexión a Internet, por lo que permite probar y desarrollar las aplicaciones de manera local e independiente.
- **Multiplataforma:** Es compatible con los principales sistemas operativos (Windows, Linux y macOS).
- **Código abierto:** Es una herramienta gratuita desarrollada por *Apache Friends*, cuyo código puede ser revisado, modificado y mejorado por la comunidad.

12. En qué casos es necesaria la instalación de la máquina virtual Java (JVM) y el software JDK en el entorno de desarrollo y en el entorno de explotación

La necesidad de instalar el software JDK o la Máquina Virtual de Java (JVM) varía dependiendo de si nos encontramos en la fase de creación del software o en su uso final. En el **entorno de desarrollo**, la instalación del JDK (*Java Development Kit*) es necesaria. Aunque técnicamente es posible escribir el código fuente en un editor de texto, el JDK proporciona el compilador (javac)

para transformar esos archivos de texto (.java) en código ejecutable o *bytecode* (.class). En esta fase no hace falta instalar la JVM por separado, ya que viene incluida de serie dentro del propio JDK para que el programador pueda probar su código. Por el contrario, en el **entorno de explotación o producción** (donde la aplicación ya está terminada y su único objetivo es ser ejecutada), el JDK es innecesario. En este caso, basta con disponer de la JVM o del JRE (*Java Runtime Environment*), el cual incluye la máquina virtual junto con las bibliotecas adicionales requeridas para que el programa funcione correctamente.

13. IDE más utilizados (características y grado de implantación actual).

Visual Studio Code (VS Code):

Características: Es un editor muy ligero, modular y gratuito. Su mayor fortaleza es su inmenso catálogo de extensiones, que permite adaptarlo e implementar casi cualquier herramienta.

Implementación: Es uno de los más utilizados a nivel mundial gracias a su enorme compatibilidad con casi todos los lenguajes de programación.

IntelliJ IDEA:

Características: Entorno extremadamente potente que destaca por su sistema inteligente de autocompletado. Cuenta con versión gratuita (*Community*) y de pago (*Ultimate*).

Implementación: Es la opción principal y más demandada en entornos empresariales y proyectos grandes basados en Java, Kotlin y el ecosistema JVM.

Eclipse:

Características: IDE clásico, gratuito, muy flexible y ampliamente extensible mediante *plugins*.

Implementación: Sigue siendo un referente en las universidades (como introducción a la programación en Java) y en empresas dedicadas al mantenimiento de software legado.

NetBeans:

Características: Ofrece una gran integración de serie con Maven y JavaFX.

Implementación: Es muy útil para el desarrollo web (soporta muy bien Java, PHP y C/C++), aunque actualmente ha quedado un paso por detrás del resto de IDEs en cuanto a innovación y número de usuarios.

14. Servidores HTTP /HTTPS más utilizados (características y grado de implantación actual).

Nginx:

Características: Muy ligero y diseñado para ofrecer el máximo rendimiento. Su configuración se basa en bloques y directivas.

Implementación: Es el rey actual para sitios web de alto tráfico. Resulta ideal para actuar como proxy inverso, balanceador de carga o en arquitecturas de microservicios.

Apache HTTP Server:

Características: Extremadamente robusto, estable y flexible gracias a su arquitectura modular (soporta casi cualquier módulo). Como contrapartida, es más pesado que Nginx.

Implementación: Mantiene una presencia masiva en servicios de *hosting* compartido y en entornos empresariales más tradicionales.

Caddy:

Características: Servidor moderno enfocado en la simplicidad. Ofrece seguridad por defecto y requiere una configuración mínima mediante una sintaxis declarativa muy sencilla.

Microsoft IIS (Internet Information Services):

Características: Integrado de forma nativa en Windows Server. Ofrece una administración visual muy intuitiva mediante interfaz gráfica (GUI).

Implementación: Altamente utilizado en entornos corporativos alineados con el ecosistema de Microsoft.

15. Apache HTTP vs Apache Tomcat

Apache HTTP (Servidor Web):

Función principal: Diseñado para servir páginas web y archivos estáticos (HTML, CSS, imágenes, scripts) que no necesitan procesamiento en el servidor.

Ventajas: Maneja muy bien un gran volumen de peticiones y es altamente configurable a través de módulos (ideal para balanceo de carga y seguridad).

Desventaja: No incluye de serie una Máquina Virtual de Java (JVM), por lo que no puede ejecutar aplicaciones Java por sí mismo.

Apache Tomcat (Servidor de Aplicaciones):

Función principal: Diseñado específicamente para alojar y ejecutar aplicaciones basadas en Java (como *servlets* y páginas JSP), ya que funciona apoyándose en una JVM.

Desventaja: Aunque técnicamente puede servir contenido estático, es mucho menos eficiente y rápido que Apache HTTP para desempeñar esa labor.

16. Navegadores HTTP /HTTPS más utilizados (características y grado de implantación actual).

Google Chrome:

Motor: Blink (basado en Chromium).

Características: Es el líder absoluto desde hace casi dos décadas. Muy rápido, totalmente integrado con el ecosistema de Google y considerado el estándar principal para el desarrollo web.

Microsoft Edge:

Motor: Blink (basado en Chromium).

Características: El exitoso sucesor de Internet Explorer. Al estar integrado de forma nativa con Windows y Microsoft 365, ha experimentado un gran crecimiento en entornos corporativos.

Mozilla Firefox:

Motor: Gecko.

Características: Es la principal alternativa de código abierto. Aunque su cuota de mercado es menor, tiene un fuerte enfoque en la privacidad y es muy valorado por los desarrolladores.

Safari:

Motor: WebKit.

Características: Es el navegador predeterminado y exclusivo de los dispositivos Apple (macOS e iOS). Destaca por su excelente optimización y rendimiento energético en baterías de portátiles y móviles.

Opera:

Características: Considerado un navegador de nicho, se diferencia por incluir herramientas integradas muy útiles, como una VPN gratuita y limitadores de consumo de recursos (CPU, memoria RAM y ancho de banda).